

使用應用程式動作，將 Android 應用程式整合至 Google 助理

來源：<https://codelabs.developers.google.com/codelabs/appactions?hl=zh-tw>

步驟數：6

瞭解基本概念，瞭解如何使用應用程式動作內建意圖，將 Android 應用程式擴充至 Google 助理，進而啟用語音功能。

目錄

1. [總覽](#)
2. [瞭解運作方式](#)
3. [準備開發環境](#)
4. [新增「開始運動」BII 功能](#)
5. [新增停止運動 BII 功能](#)
6. [後續步驟](#)

1. 總覽

透過應用程式動作，你可以使用 Google 助理直接進入應用程式功能，並透過語音完成工作。Android 開發人員會實作功能元素，新增應用程式動作。功能可讓 Google 助理瞭解哪些應用程式功能支援使用者語音要求，以及您希望如何執行這些要求。

本程式碼研究室涵蓋使用應用程式動作開發的初學者概念。如要完成本程式碼研究室，您應具備 Android 應用程式開發和 [Android Intent](#) 的經驗。如果您是 Android 新手，建議先從 [Android 開發人員基礎知識](#) 的程式碼研究室開始。

建構項目

在本程式碼研究室中，您會將兩個應用程式動作內建意圖 (BII) 新增至範例健身 Android 應用程式，讓使用者透過語音啟動及停止運動計時器。

課程內容

您將瞭解如何使用「健康與健身」類別的 BII，將 Google 助理擴充至 Android 應用程式。您也會學到如何使用 Android Studio 的 [Google 助理外掛程式](#) 測試 BII。

必要條件

繼續操作前，請確認環境中備妥下列工具：

- 已安裝 [git](#) 的終端機，可執行 Shell 指令。
- 最新版 [Android Studio](#)。
- 可存取 [Google Play 管理中心] 的 Google 帳戶。
- 可存取 Play 商店的實體或虛擬 Android 裝置。

繼續操作前，請確認您 [登入 Android Studio](#) 和測試裝置上的 Google 應用程式時，使用的是同一個 Google 帳戶。

2. 瞭解運作方式

應用程式動作可將使用者從 Google 助理連線至 Android 應用程式。運作方式為何？

當使用者要求 Google 助理使用您的應用程式執行工作時，Google 助理會將查詢內容與應用程式 `shortcuts.xml` XML 資源中定義的應用程式動作 `capability` 相符。

重要術語：功能元素是您在 `shortcuts.xml` 中定義的範本，用於宣告使用者可採取的動作類型，以便啟動應用程式並直接執行特定工作。詳情請參閱 [Android 功能說明文件](#)。

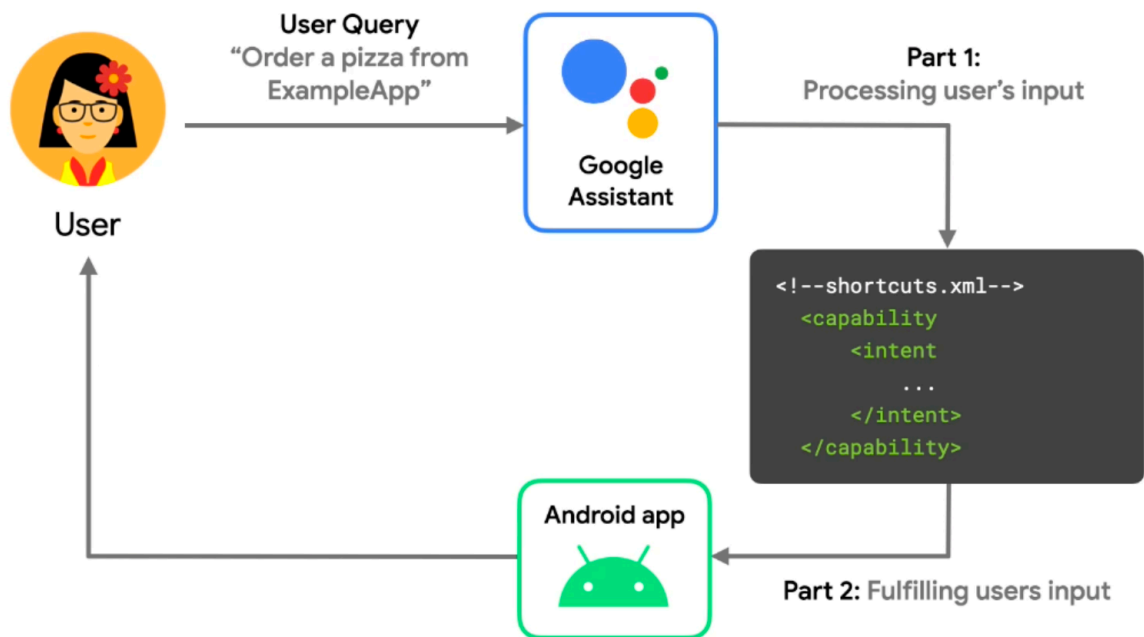


圖 1. 流程圖：說明 Google 助理如何處理應用程式動作語音查詢。

每個功能元素都會定義：

- **意圖**：應觸發功能的應用程式動作語音意圖。
- **一或多個執行要求**：Google 助理產生的 Android 意圖或深層連結，用於啟動應用程式並執行使用者語音要求。執行要求定義會指定使用者查詢中預期的參數，以及這些參數應如何編碼至啟動指令。

意圖

在自然語言理解 (NLU) 中，意圖是指一組意義相似的使用者片語。Google 提供了數十種「內建」意圖 (BII)，涵蓋各種要求類型，可搭配應用程式動作使用。舉例來說，Google 助理經過訓練後，會將「訂披薩」或「顯示甜點菜單」等詞組與 `ORDER_MENU_ITEM` BII 建立關聯。透過應用程式動作，您可以運用這些 BII，快速將常見的語音要求擴充至應用程式功能。

執行要求

當使用者要求在 `shortcuts.xml` 中觸發應用程式動作時，Android 活動必須偵測及處理傳入的 Android intent 或深層連結，並提供使用者所需的功能。這樣一來，使用者就能透過語音操作，只要提出查詢，Google 助理就會叫用您的應用程式。

式。

步驟 3 · 約 12 分鐘

3. 準備開發環境

本程式碼研究室使用 Android 適用的 Fitness 範例應用程式。使用者可以透過這款應用程式啟動及停止運動計時器，並查看運動常規的統計資料。

下載基礎檔案

如要取得本程式碼研究室的基礎檔案，請執行下列指令來複製 [GitHub 存放區](#)：

```
git clone --branch codelab-start https://github.com/actions-on-google/appactions-fitness-kotlin.git
```

複製存放區後，請在 Android Studio 中開啟：

1. 在「Welcome to Android Studio」對話方塊中，按一下「Import project」。
2. 找出並選取複製存放區的資料夾。

注意：上述 `git clone` 指令會簽出存放區的 `codelab-start` 分支，這是本程式碼研究室的起點。如要搶先查看完成程式碼研究室指示後的應用程式，請查看存放區的[主要分支版本](#)。

更新 Android 應用程式 ID

更新應用程式的應用程式 ID，即可在測試裝置上清楚識別應用程式，並避免在將應用程式上傳至 Play 管理中心時發生「套件名稱重複」錯誤。如要更新應用程式 ID，請開啟 `app/build.gradle`：

```
android {  
    ...  
    defaultConfig {  
        applicationId "com.MYUNIQUENAME.android.fitactions"  
        ...  
    }  
}
```

將 `applicationId` 欄位中的「MYUNIQUENAME」替換為專屬名稱。

注意：更新 `applicationId` 後，Android Studio 會顯示以下警告：「Gradle files have changed since last project sync...」（自上次專案同步處理後，Gradle 檔案已變更...）。在這個警告方塊中，按一下「立即同步處理」，以免上傳應用程式時發生問題。

在裝置上試用應用程式

進一步變更應用程式的程式碼前，建議先瞭解範例應用程式的功能。在開發環境中測試應用程式的步驟如下：

1. 開啟虛擬或實體 Android 測試裝置。
2. 確認 Google 助理應用程式是否正常運作。

3. 使用 Android Studio 在裝置上部署及執行範例應用程式。

如要測試應用程式，請按照下列步驟操作：

1. 在 Android Studio 中，依序選取「Run」>「Run app」，或按一下工具列中的「Run」圖示



。

2. 如果使用虛擬裝置，請在「Select Deployment Target」對話方塊中選取虛擬裝置，然後按一下「OK」。建議使用的 OS 版本為 Android 8 (API 級別 26) 以上，不過動作最多可以支援到 Android 5 (API 級別 21) 的裝置。

3. 開啟應用程式後，長按主畫面按鈕設定 Google 助理，並確認是否正常運作。登入 Google 助理 (如果尚未登入)。

4. 重新開啟應用程式。

注意：如要進一步瞭解虛擬裝置，請參閱「[建立及管理虛擬裝置](#)」。

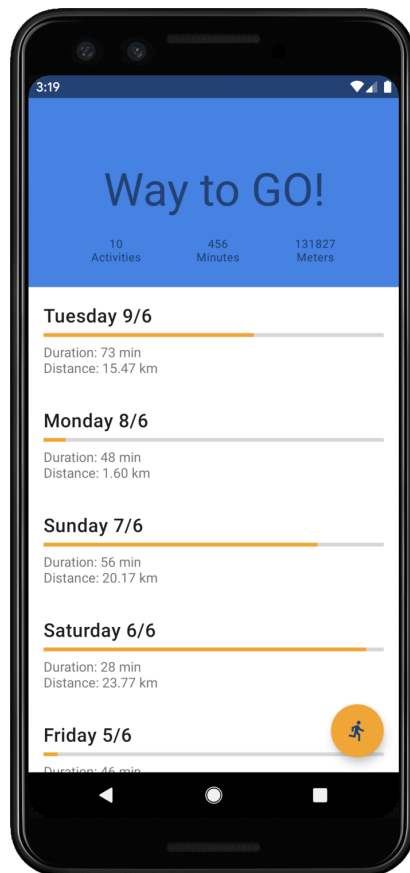


圖 2. 顯示運動統計資料的 Fit Actions 範例應用程式。

簡單瀏覽應用程式，瞭解其功能。輕觸「跑步」圖示可啟動運動計時器，輕觸「X」圖示則可停止計時器。這些是您透過應用程式動作啟用語音控制的任務。

安裝 Google 助理外掛程式

[Google 助理外掛程式](#)可讓您在測試裝置上測試應用程式動作。如要在 Android Studio 中新增這項功能，請按照下列步驟操作：

1. 依序前往「File」>「Settings」(MacOS 中則為「Android Studio」>「Preferences」)。

2. 在「外掛程式」部分，前往「市集」並搜尋「Google 助理」。

3. 安裝工具，然後重新啟動 Android Studio。

注意：測試外掛程式會透過 Android 裝置上的 Google 應用程式，將資訊傳送給 Google 助理。測試裝置必須能存取 Play 商店，才能更新 Google 應用程式，應用程式動作才能正常運作。

4. 新增「開始運動」 BII 功能

使用者可以透過 `actions.intent.START_EXERCISE` BII 開啟應用程式並以語音開始運動。在這個步驟中，您會為這個 BII 實作功能，讓使用者要求 Google 助理在「健身」應用程式中開始跑步。

定義功能

Google 助理會使用 `shortcuts.xml` 中定義的 `capability` 元素，透過下列步驟處理語音指令：

1. Google 助理會將使用者語音查詢與應用程式功能中定義的 BII 進行比對。
2. Google 助理會從查詢中擷取值，並填入 BII 參數。每個參數都會新增至附加至所產生 `Intent` 的 `Bundle`。
3. Google 助理會使用 `Intent` 啟動應用程式，並授予應用程式存取已組合參數的權限。

`START_EXERCISE` BII 支援 `exercise.name` BII 參數。您將使用這個參數，讓使用者指定要在應用程式中追蹤的運動類型。

將 `START_EXERCISE` BII 新增至應用程式，方法是在 `app/src/main/res/xml` 範例專案目錄的 `shortcuts.xml` 中加入這個 `capability`：

```
<!-- shortcuts.xml -->

<capability android:name="actions.intent.START_EXERCISE">
  <intent
    android:action="android.intent.action.VIEW"
    android:targetPackage="PUT_YOUR_APPLICATION_ID_HERE"
    android:targetClass="com.devrel.android.fitactions.FitMainActivity">
    <parameter
      android:name="exercise.name"
      android:key="exerciseType"/>
    </intent>
  </capability>
```

將 `PUT_YOUR_APPLICATION_ID_HERE` 換成您在上一個步驟定義的專屬 `applicationId`。

上述範例 XML：

- 宣告 `START_EXERCISE` BII 的功能。
- 指定 Google 助理產生的 Android `intent`，用於啟動應用程式：
 - `targetPackage` 和 `targetClass` 屬性會指定接收活動。
 - `parameter` 屬性會將 `exercise.name` BII 參數對應至活動收到的 `Bundle` 額外內容中的 `exerciseType`。

使用內嵌清查處理 BII 參數

BII 參數代表從 Google 助理使用者查詢中擷取的元素。舉例來說，當使用者說出「Ok Google，使用範例應用程式開始跑步」時，Google 助理會將「跑步」擷取到 `exercise.name` [schema.org](https://developers.google.com/bii/schema.org) BII 參數中。對於部分 BII，您可以指示 Google 助理將 BII 參數與應用程式預期的 ID 集相符。

方法是將內嵌清查元素繫結至 BII 參數。內嵌清查是一組支援的 BII 參數值，例如「跑步」、「健行」和「慢跑」，以及對應的快速指令 ID，例如 `EXERCISE_RUN`。透過這項清查繫結，Google 助理可將相符參數的快速鍵 ID 傳遞至執行要求活動，而非原始查詢值。

部分 BII 參數 (例如 `exercise.name`) 需要內嵌目錄才能運作。如要處理這項參數，請將下列目錄 `shortcut` 元素新增至 `shortcuts.xml`：

```
<!-- shortcuts.xml -->

<shortcuts>
  <shortcut
    android:shortcutId="running"
    android:shortcutShortLabel="@string/activity_running">
    <capability-binding android:key="actions.intent.START_EXERCISE">
      <parameter-binding
        android:key="exercise.name"
        android:value="@array/runningSynonyms"/>
    </capability-binding>
  </shortcut>

  <shortcut
    android:shortcutId="walking"
    android:shortcutShortLabel="@string/activity_walking">
    <capability-binding android:key="actions.intent.START_EXERCISE">
      <parameter-binding
        android:key="exercise.name"
        android:value="@array/walkingSynonyms"/>
    </capability-binding>
  </shortcut>

  <shortcut
    android:shortcutId="cycling"
    android:shortcutShortLabel="@string/activity_cycling">
    <capability-binding android:key="actions.intent.START_EXERCISE">
      <parameter-binding
        android:key="exercise.name"
        android:value="@array/cyclingSynonyms"/>
    </capability-binding>
  </shortcut>

  <capability> ... </capability>
</shortcuts>
```

在上述程式碼中，您定義了三個捷徑，代表應用程式支援的運動類型 (跑步、步行和騎自行車) 的內嵌清查。每個快速鍵都會以以下方式繫結至功能：

- 每個 `capability-binding` 元素的 `android:key` 屬性都參照為功能定義的相同 `START_EXERCISE` BII。
- 每個快速鍵的 `parameter-binding` 元素都會對應至 `exercise.name` BII 參數。

注意：完成這個步驟後，Android Studio 可能會警告您無法解析同義字值的符號。完成下一個步驟即可解決這個問題。

新增內嵌目錄同義詞

前述商品型錄捷徑中 `parameter-binding` 元素的 `android:value` 屬性，是指每個商品型錄元素的同義字陣列資源。同義詞可啟用元素類型的變體，例如「跑步」、「慢跑」和「衝刺」可參照相同的 `shortcutId`。在專案的 `array.xml` 資源中新增下列同義字項目：

```
<!-- array.xml -->
<array name="runningSynonyms">
    <item>Run</item>
    <item>Jog</item>
    <item>Jogging</item>
    <item>Sprint</item>
</array>

<array name="walkingSynonyms">
    <item>Walk</item>
    <item>Hike</item>
    <item>Hiking</item>
</array>

<array name="cyclingSynonyms">
    <item>Biking</item>
    <item>Riding</item>
    <item>Pedaling</item>
</array>
```

執行傳入的 Android 意圖

Android 意圖是 Android 用來要求其他應用程式執行動作的訊息物件。Google 助理會根據觸發功能中的設定詳細資料產生意圖，藉此完成使用者的語音查詢。如要執行 `START_EXERCISE` 功能的意圖，請更新 `FitMainActivity` 目標類別，處理傳入的意圖和 `BII` 參數。

首先，請將 `Intent.handleIntent` 函式替換為下列程式碼：

```
//FitMainActivity.kt

private fun Intent.handleIntent() {
    when (action) {
        // When the BII is matched, Intent.Action_VIEW will be used
        Intent.ACTION_VIEW -> handleIntent(data)
        // Otherwise start the app as you would normally do.
        else -> showDefaultView()
    }
}
```

接著，使用下列程式碼將新的 `handleIntent` 函式新增至類別：

```
//FitMainActivity.kt

/**
 * Use extras provided by the intent to handle the different BIIs
 */

private fun handleIntent(data: Uri?) {
    // path is normally used to indicate which view should be displayed
    // i.e https://fit-actions.firebaseio.com/start?exerciseType="Running" -> path = "start"
    var actionHandled = true

    val startExercise = intent?.extras?.getString(START_EXERCISE)
    // Add stopExercise variable here

    if (startExercise != null){
        val type = FitActivity.Type.find(startExercise)
        val arguments = Bundle().apply {
            putSerializable(FitTrackingFragment.PARAM_TYPE, type)
        }
        updateView(FitTrackingFragment::class.java, arguments)
    } // Add conditional for stopExercise
    else{
        // path is not supported or invalid, start normal flow.
        showDefaultView()

        // Unknown or invalid action
        actionHandled = false
    }
    notifyActionSuccess(actionHandled)
}
```

在上述 `Intent.handleIntent` 函式中，當 `ACTION_VIEW` 觸發時，應用程式動作意圖資料會傳遞至 `handleIntent` 函式。`START_EXERCISE` 意圖中綁定的 BII 參數會透過 `intent?.extras?.getString(START_EXERCISE)` 存取。函式的其餘部分會更新 `FitTrackingFragment`，顯示所選的 `startExercise` 健身類型。

測試應用程式動作

在開發應用程式動作時，您可以使用 [Google 助理外掛程式](#)，在測試裝置上預覽動作。您也可以使用外掛程式調整動作的意圖參數值，測試應用程式如何處理使用者以各種方式向 Google 助理提出應用程式要求。

如要使用外掛程式測試應用程式動作，請按照下列步驟操作：

1. 在 Android Studio 中，依序選擇「Run」>「Run App」，或按一下頂端工具列的「Run」圖示，即可執行應用程式。
2. 依序前往「Tools」>「App Actions」>「Google Assistant」>「App Actions Test Tool」。
3. 按一下「Create Preview」。如果系統要求，請詳閱並接受應用程式動作政策和服務條款。
4. 選取 `actions.intent.START_EXERCISE` 內建意圖。
5. 保留「運動」方塊的預設「跑步」值。
6. 按一下「執行應用程式動作」。確認 Google 助理可深層連結至應用程式的運動計時器，且計時器已開始跑步類型的運動。

提示：如果無法順利執行測試工具，請參閱[疑難排解指南](#)。

您已使用 `START_EXERCISE` BII 實作第一個應用程式動作。恭喜！接著，我們會讓使用者在應用程式中停止進行中的運動。

5. 新增停止運動 BII 功能

使用者可以透過 `actions.intent.STOP_EXERCISE` BII 停止健身活動，例如說出「Ok Google，停止範例應用程式的跑步活動」。在 Fitness 應用程式中實作這項 BII，方法是在 `shortcuts.xml` 中新增第二個 `capability`：

```
<!-- shortcuts.xml -->

<capability android:name="actions.intent.STOP_EXERCISE">
  <intent
    android:action="android.intent.action.VIEW"
    android:targetPackage="PUT_YOUR_APPLICATION_ID_HERE"
    android:targetClass="com.devrel.android.fitactions.FitMainActivity">
    <!-- Eg. name = "Running" -->
    <parameter
      android:name="exercise.name"
      android:key="stopExercise"/>
  </intent>
</capability>
```

將 `PUT_YOUR_APPLICATION_ID_HERE` 替換成專屬 `applicationId`。

使用內嵌清查處理 BII 參數

這個 BII 支援與 `START_EXERCISE` BII 相同的 `exercise.name` 參數，可讓使用者指定要結束的進行中訓練。如要啟用這項功能，請在 `shortcuts.xml` 中新增第二組商品目錄捷徑元素：

```

<!-- shortcuts.xml -->

<shortcut
    android:shortcutId="running"
    android:shortcutShortLabel="@string/activity_running">
    <capability-binding android:key="actions.intent.STOP_EXERCISE">
        <parameter-binding
            android:key="exercise.name"
            android:value="@array/runningSynonyms" />
        </capability-binding>
    </shortcut>

<shortcut
    android:shortcutId="walking"
    android:shortcutShortLabel="@string/activity_walking">
    <capability-binding android:key="actions.intent.STOP_EXERCISE">
        <parameter-binding
            android:key="exercise.name"
            android:value="@array/walkingSynonyms" />
        </capability-binding>
    </shortcut>

<shortcut
    android:shortcutId="cycling"
    android:shortcutShortLabel="@string/activity_cycling">
    <capability-binding android:key="actions.intent.STOP_EXERCISE">
        <parameter-binding
            android:key="exercise.name"
            android:value="@array/cyclingSynonyms" />
        </capability-binding>
    </shortcut>

```

執行傳入的 Android 意圖

更新 `FitMainActivity` 類別，讓應用程式處理傳入的 `STOP_EXERCISE` Android Intent。首先，在 `handleIntent` 函式中新增變數，以保留 `STOP_EXERCISE` 意圖資料：

```

// FitMainActivity.kt

private fun handleIntent(data: Uri?) {
    val stopExercise = intent?.extras?.getString(STOP_EXERCISE)
    //...
}

```

接著，更新 `handleIntent` 函式的條件邏輯，以處理 `STOP_EXERCISE` 意圖：

```
// FitMainActivity.kt

private fun handleIntent(data: Uri?) {
    //...
    if (startExercise != null){
        val type = FitActivity.Type.find(startExercise)
        val arguments = Bundle().apply {
            putSerializable(FitTrackingFragment.PARAM_TYPE, type)
        }
        updateView(FitTrackingFragment::class.java, arguments)
    } // Add conditional for stopExercise
<strong>
} else if(stopExercise != null){
    // Stop the tracking service if any and return to home screen.
    stopService(Intent(this, FitTrackingService::class.java))
    updateView(FitStatsFragment::class.java)
}
</strong>
//...
}
```

在上述程式碼中，您更新了 `handleIntent` 函式，檢查傳入的 Android Intent 中是否有 `STOP_EXERCISE` BII。如果找到，這項函式會停止作用中的計時器，並將使用者帶回主畫面。

測試應用程式動作

如要使用 Google 助理外掛程式測試應用程式動作，請按照下列步驟操作：

1. 在 Android Studio 中，依序選擇「Run」>「Run App」，或按一下頂端工具列的「Run」圖示，即可執行應用程式。
2. 在應用程式中，開始新的「跑步」運動。
3. 在 Android Studio 中開啟外掛程式：依序前往「Tools」>「App Actions」>「Google Assistant」>「App Actions Test Tool」。
4. 按一下「Create Preview」。
5. 選取 `actions.intent.STOP_EXERCISE` 內建意圖。
6. 保留「運動」方塊的預設「跑步」值。
7. 按一下「執行應用程式動作」。確認 Google 助理停止運動，並將你帶回主畫面。

提示：如果無法順利執行測試工具，請參閱[疑難排解指南](#)。

6. 後續步驟

恭喜！

您現在瞭解如何使用 Google 助理內建意圖，為 Android 應用程式啟用語音功能。在本程式碼研究室中，您已瞭解以下內容：

- 如何讓使用者透過 Google 助理深入瞭解特定應用程式功能。
- 如何使用內嵌廣告空間。
- 如何使用 [Google 助理外掛程式](#) 測試 BII。

後續步驟

現在，您可以嘗試進一步調整健身應用程式。如要參考完成的專案，請參閱 GitHub 上的[主要分支版本](#)。

以下提供一些建議，協助您進一步瞭解如何使用應用程式動作擴充這個應用程式：

- 請參閱其他[應用程式動作程式碼研究室](#)。
- 請參閱應用程式動作的[內建意圖參考資料](#)，瞭解更多可將應用程式擴展至 Google 助理的 BII。

如要繼續使用 Google 助理開發動作，請參閱下列資源：

- developers.google.com/assistant：Actions on Google 的官方說明文件網站。
- [應用程式動作範例索引](#)：範例應用程式和程式碼，可探索應用程式動作功能。
- [Actions on Google GitHub 存放區](#)：範例程式碼和程式庫。
- [r/GoogleAssistantDev](#)：專為使用 Google 助理開設的官方 Reddit 社群。

歡迎在 Twitter 追蹤 [@ActionsOnGoogle](#)，隨時掌握最新公告，並在推文中加上 [#appactions](#)，分享您開發的內容！

意見回饋問卷調查

最後，請填寫這份[問卷](#)，提供您對本程式碼研究室的意見。
